

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

NoSQL

Лабораторная работа №1

Вариант №1865727

Выполнили:

Яруков Артём Дмитриевич

Лежнев Никита Сергеевич

Проверил:

Преподаватель

Цю Тяньшэн

Санкт-Петербург
2026 г.

Содержание

1	Текст задания	2
2	Исходный код проекта	3
3	Реализованные механизмы	3
4	Исследование поведения системы: Перезапуск узла кластера	3
4.1	Конфигурация кластера Riak KV	3
4.2	Сценарий 1: Перезапуск узла при живых соседях	3
4.3	Сценарий 2: Отказ узлов, принявших новую запись	6
4.4	Сценарий 3: Экстремальный отказ (2 узла из 3) и механизм Sloppy Quorum	8
5	Выводы о применимости Riak KV	9

1 Текст задания

Вариант №1865727

Разработать веб-приложение на языке Java с использованием Spring Boot, реализующее систему уведомлений.

- **Предметная область:** доставка еды.
- **Основной объект системы:** пользователь.
- **Пользовательская роль:** оператор.
- **Обязательный сценарий:** просмотр истории действий.
- **Система хранения типа «ключ — значение»:** Riak KV.

Дополнительные параметры

- **Сценарий временного хранения:** временная пользовательская сессия;
- **Сценарий кэширования:** профиль пользователя;
- **Атомарный механизм:** счётчик посещений страницы;
- **Исследование поведения системы:** перезапуск одного узла.

Требования к программе

1. Развернуть систему хранения, указанную в варианте.
2. Реализовать подключение к ней из Java/Spring Boot.
3. Выбрать и обосновать способ представления данных в модели «ключ — значение».
4. Реализовать хранение части данных приложения в выбранной системе.
5. Реализовать механизм временных данных: TTL для Redis, application-level expiration для Riak KV или lease для Etcd.
6. Реализовать кэширование часто запрашиваемых данных.
7. Реализовать назначенный атомарный счётчик, ограничение запросов или блокировку.
8. Настроить сохранение и восстановление данных, если выбранная система поддерживает этот механизм.
9. Исследовать назначенный сценарием режим поведения системы.
10. Сделать выводы о применимости выбранного хранилища для разработанного приложения.

2 Исходный код проекта

- Репозиторий GitHub: <https://github.com/artyomyarukov/nosqllab1>

3 Реализованные механизмы

1. **Application-level Expiration:** Объект сессии содержит временную метку `expiresAt` (время жизни 15 минут). При каждом чтении проверяется `isExpired()`. Дополнительно реализован активный фоновый уборщик Spring с аннотацией `@Scheduled(fixedRate = 60000)`, сканирующий корзину раз в минуту и физически удаляющий просроченные токены.
2. **Кэширование :** Реализована проверка кэша перед обращением к PostgreSQL. При редактировании профиля выполняется событийно-ориентированная инвалидация ключа в Riak.
3. **Атомарный счётчик:** Инкремент просмотров осуществляется через операцию `CounterUpdate`, что предотвращает состояние гонки.

4 Исследование поведения системы: Перезапуск узла кластера

4.1 Конфигурация кластера Riak KV

Кластер развернут в изолированной сети Docker Compose со статическими адресами:

- `riak1` — 172.23.0.11
- `riak2` — 172.23.0.12
- `riak3` — 172.23.0.13

```
PS C:\Users\ppjis\nosqlab1> docker exec -it riak1 riak-admin member-status
===== Membership =====
Status      Ring      Pending   Node
-----
valid       34.4%     --        'riak@172.23.0.11'
valid       32.8%     --        'riak@172.23.0.12'
valid       32.8%     --        'riak@172.23.0.13'
-----
Valid:3 / Leaving:0 / Exiting:0 / Joining:0 / Down:0
PS C:\Users\ppjis\nosqlab1>
```

Рис. 1: Штатное состояние кластера до начала испытаний

4.2 Сценарий 1: Перезапуск узла при живых соседях

Ход эксперимента

1. В штатном режиме на узел отправлялись операции записи .

2. Выполнена принудительная остановка узла `riak2` командой `docker stop riak2`.
3. Проверено состояние кластера `riak1` (`riak-admin ring-status`).
4. Во время изоляции узла клиент продолжил совершать запросы на чтение и запись через Spring Boot.
5. Произведен запуск узла командой `docker start riak2`.
6. Проверено физическое наличие переданных данных на узле `riak2` через прямой HTTP-запрос.

```
PS C:\Users\ppjis\nosqlab1> docker exec -it riak1 riak-admin ring-status
===== Claimant =====
Claimant: 'riak@172.23.0.11'
Status:    up
Ring Ready: true

===== Ownership Handoff =====
No pending changes.

===== Unreachable Nodes =====
The following nodes are unreachable: ['riak@172.23.0.12']

WARNING: The cluster state will not converge until all nodes
are up. Once the above nodes come back online, convergence
will continue. If the outages are long-term or permanent, you
can either mark the nodes as down (riak-admin down NODE) or
forcibly remove the nodes from the cluster (riak-admin
force-remove NODE) to allow the remaining nodes to settle.
PS C:\Users\ppjis\nosqlab1>
```

Рис. 2: Фиксация недоступности узла `riak2` через `ring-status` (Nodes that are down)

Наблюдаемое поведение («Перетекание данных»)

- Клиентское приложение не получило ошибок. Пул соединений Spring Boot автоматически перенаправил операции на доступные узлы `riak1` и `riak3` (рис. 3).
- После подачи команды `docker start riak2` ноды `riak1` и `riak3` зафиксировали появление соседа в сети и асинхронно передали (рис. 4).
- **Верификация перетекания данных:** Для подтверждения того, что данные физически записались на перезапущенный узел `riak2`, выполнен прямой HTTP-запрос к ноде `riak2` на её личный порт 8099 по адресу `/buckets/action_history/keys/2` (рис. 5). Узел успешно вернул полный журнал уведомлений, доказав завершение процесса передачи данных.

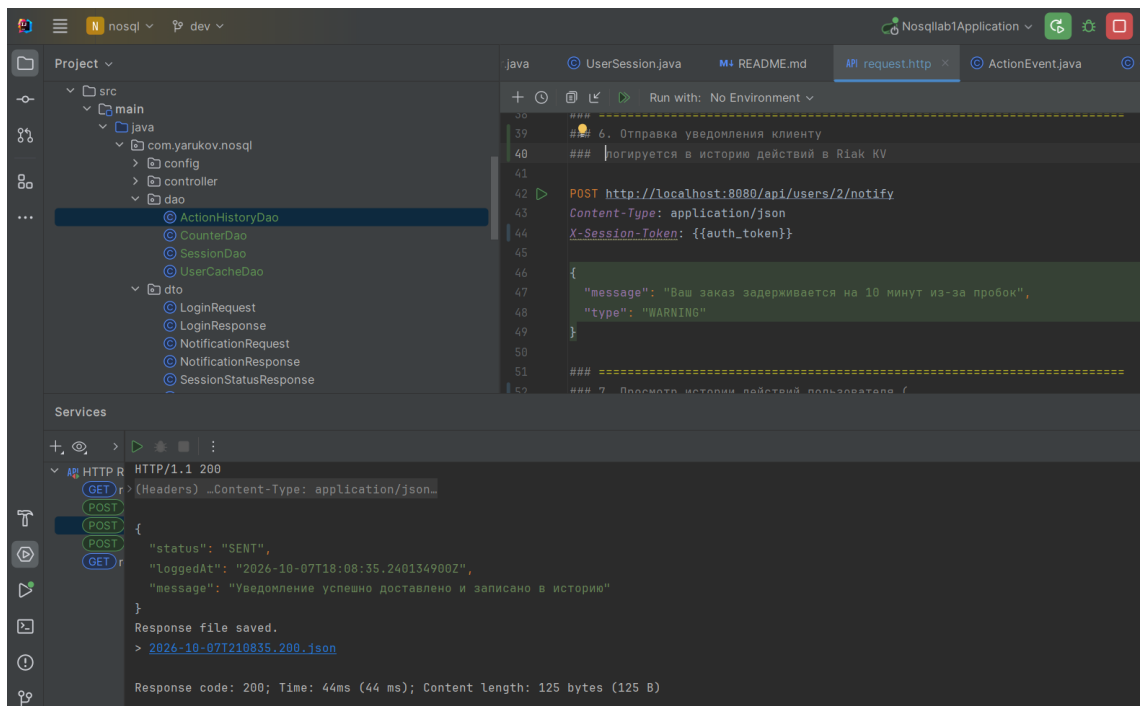


Рис. 3: Успешная обработка запросов (HTTP 200 OK) при отключенном узле riak2

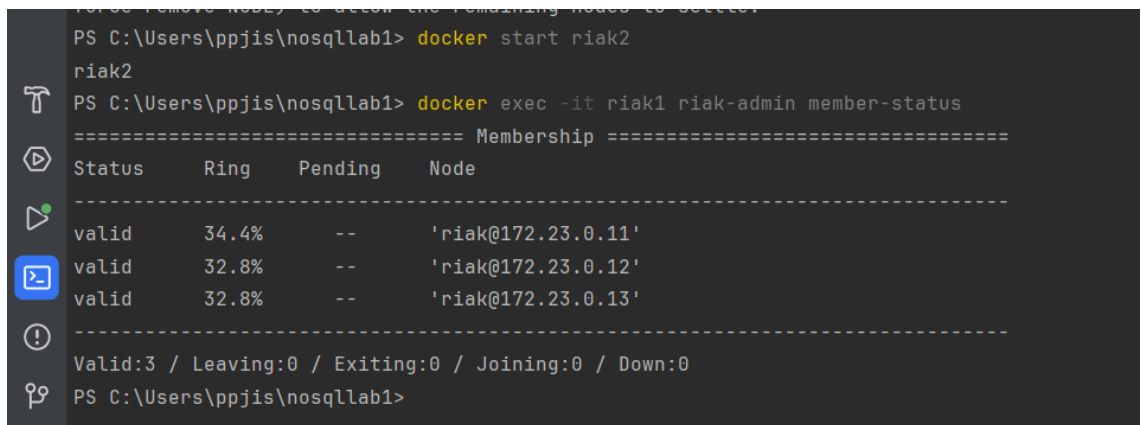


Рис. 4: Восстановление узла riak2

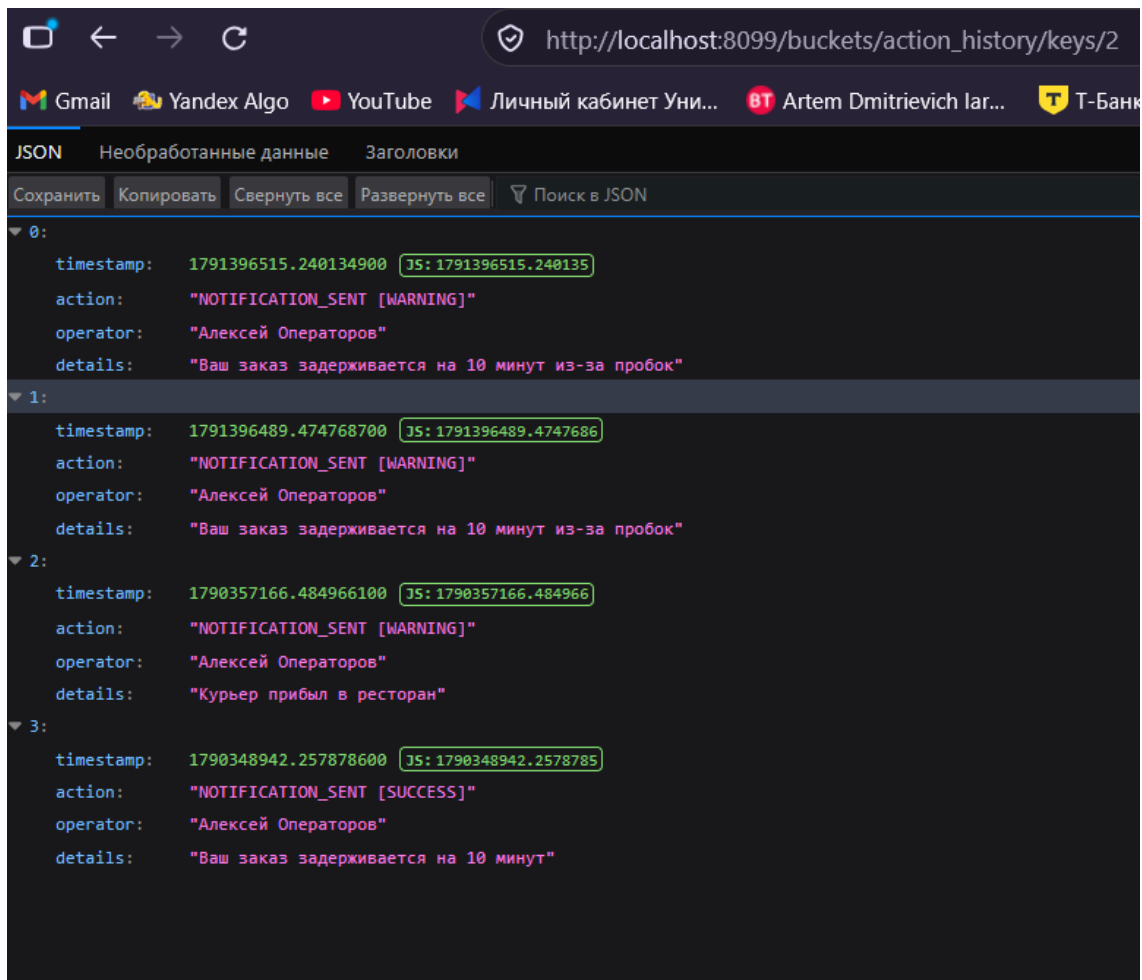


Рис. 5: Прямое чтение из узла riak2 (порт 8099): подтверждение физического перетекания данных

4.3 Сценарий 2: Отказ узлов, принявших новую запись

Ход эксперимента

1. В момент, когда узел **riak2** был остановлен, через API отправлено новое уведомление. Запись успешно сохранена кворумом узлов **riak1** и **riak3**. Узел **riak2** находится в неведении о новой записи.
2. Остановлен узел **riak3** (`docker stop riak3`), одновременно запущен ранее изолированный узел **riak2** (`docker start riak2`).
3. В топологии сети остались только узлы **riak1** (обладающий свежей версией) и **riak2** (обладающий устаревшей версией). Узел **riak3** выведен из строя.
4. Через клиентский интерфейс выполнен запрос на чтение истории действий (`GET /api/users/2/history`).
5. Выполнена верификация починки узла **riak2** через прямой запрос к его порту 8099.

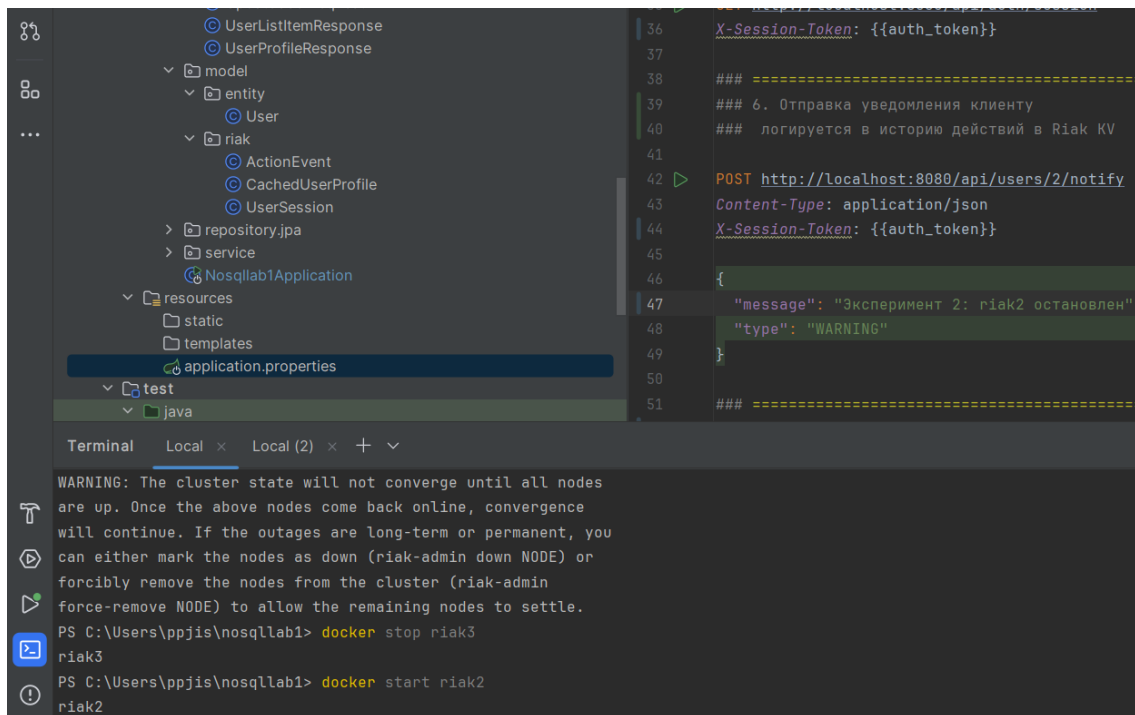


Рис. 6: Терминал: остановка узла riak3 со свежими данными и запуск отставшего узла riak2

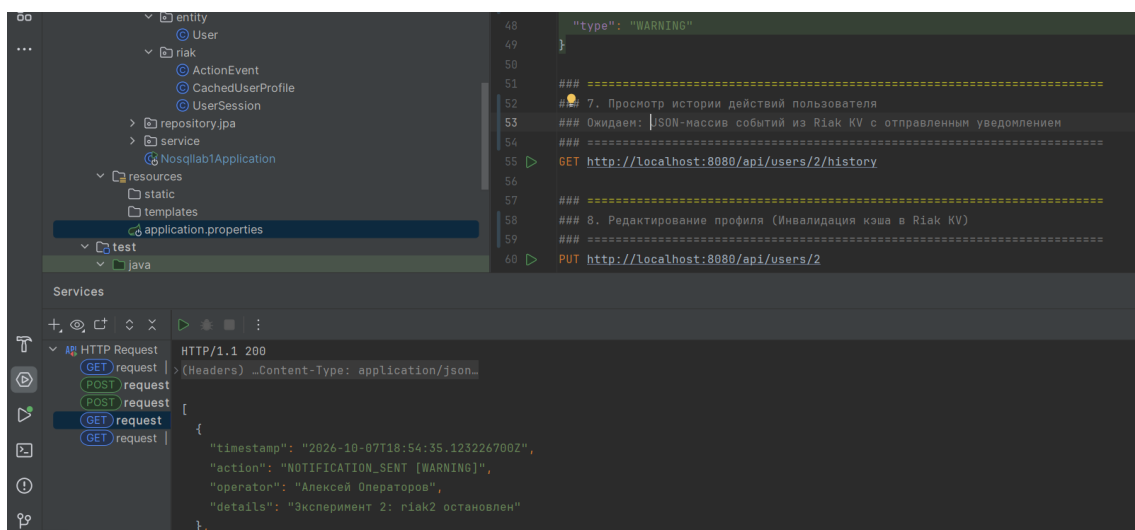


Рис. 7: Успешная отдача актуальной истории

Наблюдаемое поведение

- При поступлении запроса на чтение сформирован кворум между узлами riak1 и riak2.
- **Механизм Read Repair:** В процессе обработки чтения координатор riak1 в фоновом асинхронном режиме передал недостающую запись на узел riak2.
- При проверке состояния узла riak2 через прямой HTTP-запрос (http://localhost:8099/buckets/action_history/keys/2) зафиксировано появление новой записи (рис. 8), что подтверждает факт успешного выполнения Read Repair без участия узла riak3.

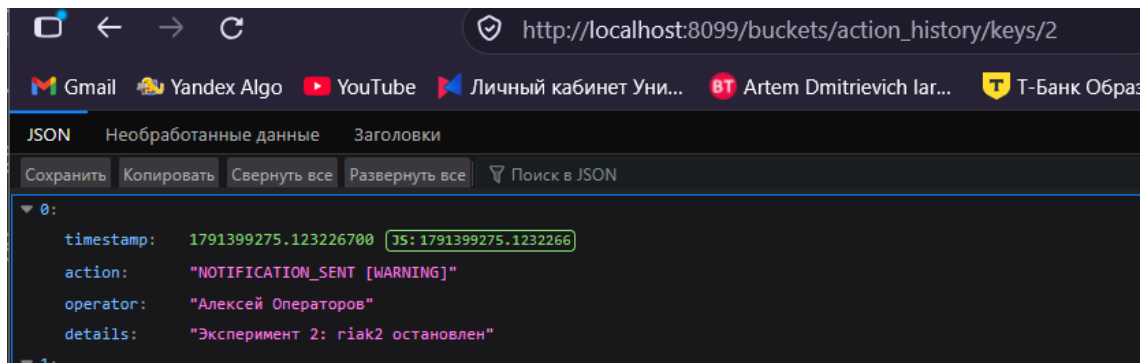


Рис. 8: Прямой запрос к riak2 (порт 8099): подтверждение успешного обновления данных

4.4 Сценарий 3: Экстремальный отказ (2 узла из 3) и механизм Sloppy Quorum

Ход эксперимента

1. Проверена реакция системы на экстремальный отказ большинства: одновременное отключение двух узлов (`docker stop riak2 riak3`).
2. В кластере остался активным единственный узел riak1 (1 узел из 3).
3. Зафиксировано состояние изоляции узла через `riak-admin ring-status` (рис. 9), где Ring Ready: false.
4. Через API выполнена операция записи нового уведомления (`POST /api/users/2/notify`).

```
PS C:\Users\ppjis\nosqlab1> docker exec -it riak2 riak-admin ring-status
===== Claimant =====
Claimant: 'riak@172.23.0.11'
Status:    down
Ring Ready: unknown

===== Ownership Handoff =====
No pending changes.

===== Unreachable Nodes =====
The following nodes are unreachable: ['riak@172.23.0.11', 'riak@172.23.0.13']
```

Рис. 9: Состояние кольца Riak при падении двух узлов: Ring Ready = false, узлы 12 и 13 unreachable

Наблюдаемое поведение (Работа Sloppy Quorum)

- **Принцип максимальной доступности (AP по теореме CAP):** В Riak сработал механизм Sloppy Quorum. Одиночный узел riak1, принял запись на себя, сохранив её с обязательством передать её узлам riak2 и riak3 после их включения.

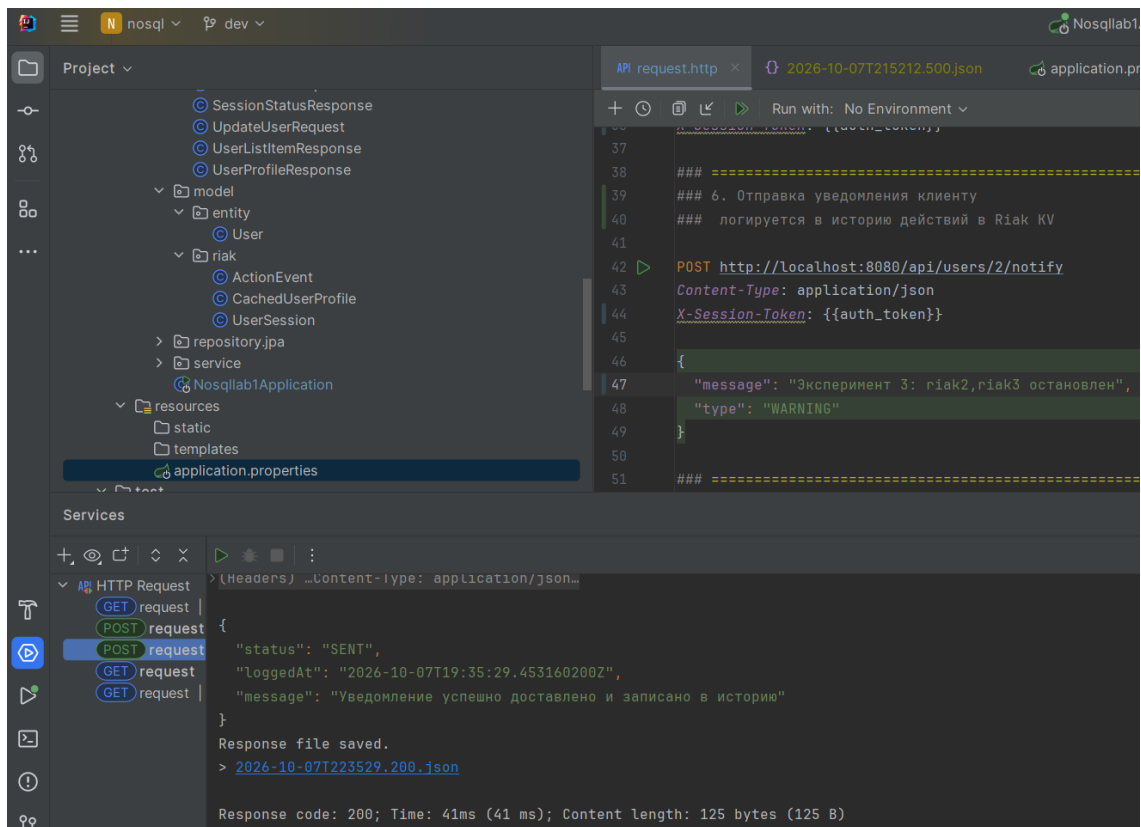


Рис. 10: Подтверждение работы Sloprry Quorum: успешная запись 200 ОК при живой 1 ноде из 3

5 Выводы о применимости Riak KV

В ходе выполнения лабораторной работы исследовано распределенное хранилище типа «ключ — значение» Riak KV в связке с реляционной СУБД PostgreSQL 17.

Отказоустойчивость и доступность (AP-система): Остановка любого узла не приводит к прерыванию обслуживания клиентов благодаря кворумам.